

CIS*3090, Fall 2025
Assignment 1
Due date: October 19, 11:59pm

1. Assignment overview

This program will give you a chance to practice some parallel programming with shared memory and Pthreads and learn to think about program performance. Since this is your first assignment, we will keep the parallel task itself simple and focus on scheduling the tasks, managing tasks, and designing/conducting performance analysis of your code.

You can complete this assignment individually or in teams of two.

2. Program description

Write a program that implements a system for queueing multiple computationally intensive tasks and executing them in parallel using Pthreads. In order to optimize the performance, we will not spawn a new thread for each task. Instead, we will maintain a **thread pool** - a set of threads that will start when the program starts. Each new task will be allocated to a free thread in the pool. If all threads are busy, the task will be queued until a thread is available. Eventually, we will run out of tasks and can then shut down the entire system.

Your program will work as follows. The main thread begins by starting a user-specified number of new threads that immediately go to sleep on a condition wait. This is our pool of worker threads. The main thread generates blocks of tasks to be carried out by the worker threads. Each time a new block of tasks is generated, the main thread places a task on the queue and awakens a worker thread in the pool with a condition signal. The awakened worker thread then grabs a task from the queue, does the work, and then goes back to sleep on a condition wait.

Your system should know how many worker threads in the pool are available. The counter would be initialized to the max number of threads when the program starts and then decremented / incremented as worker threads start / finish their tasks. You can do this manually, with a mutex-protected integer counter, or you can use a counting semaphore instead.

If the pool is busy when the main thread generates a task, none of the worker threads are waiting on the condition variable, so none of them will notice the main thread's signal after the new task is placed on the queue. Therefore, when a worker thread finishes its current work, it should check the queue first. If the queue has a work task in it, the worker thread removes the task and does the work. If the queue is empty, the worker thread goes to sleep on a condition wait. However, before it goes to sleep, it should update the global counter (it is now free).

Once there are no more tasks in the queue, and all the worker threads have completed their allocated tasks, your program should shut down. How you implement this is up to you. Just make sure that you clean up all the allocated resources and don't leak any memory.

To keep things simple, all your tasks will be the same - computation of the value of π using specific number of iterations n . The algorithm for this is very simple, and the serial version that you need is presented in Section 4.4 of your textbook. The parameter for each task placed on the queue will just be the number of iterations that the worker thread running the task will need to perform when computing the value of π .

The program will take one command-line argument - the name of the file that will contain the parameters for the program and the optional boolean output flag:

```
./A1 fileName flag
```

The file will be structured as follows:

- The first line will be the number of threads in the pool
- Each subsequent line will be the task description - i.e. the number of iterations for each π computation

For example, a file that will require 4 threads and will contain 5 tasks (5 different numbers of iterations) will look like something this:

```
4
1200000
1200000
2400000
23
34345000
```

If the value of the `flag` argument is `false` or it is omitted, the program does not display anything. This produces cleaner timings, since the `printf` calls are relatively slow and would affect the overall run time.

If the `flag` argument is `true`, each worker thread must display its results to `stdout` after it finishes its task in the following format:

Thread XX completed computed Pi using YY iterations, the result is ZZ
where XX is the thread number in the pool, YY is the number of iterations in the task it just finished, ZZ is the value of π is computed.

So, for example, the output for the file above would look something like this:

```
Thread 4 completed computed Pi using 23 iterations, the result is ZZ4
Thread 1 completed computed Pi using 1200000 iterations, the result is ZZ1
Thread 2 completed computed Pi using 1200000 iterations, the result is ZZ2
Thread 3 completed computed Pi using 2400000 iterations, the result is ZZ3
Thread 4 completed computed Pi using 34345000 iterations, the result is ZZ5
```

Keep in mind that ZZ1-5 are actual values of π , and ZZ1 should be the same as ZZ2, since we calculated π using the same number of iterations. The value of ZZ4 would be fairly inaccurate due to a very small number of iterations. Thread 4 would most likely be the first to finish, since it has so few steps. It will also likely be the one that grabs the 5th task from the queue, which is reflected in my hypothetical output above.

Create a Makefile that compiles your program with the command `make A1`. The resulting executable must be named `A1`.

3. Analysis and report

As we discussed in class, your parallel code has to

- be correct - accurately compute what it is supposed to compute without run-time errors (leaking memory, accessing uninitialized variables, etc.)

- have performance that you fully understand and (hopefully) show a speedup

3.1 Correctness analysis

Your report will need to contain the description of how you designed the test procedure for the code and verified its correctness. Your report will also need to contain instructions on how to run specific test cases you have designed. You must include all data (i.e. input files) that you developed for your test cases.

3.2 Performance analysis

In addition, your report must include the discussion of program performance. You will need to show whether your parallel program is better (or worse!) than the serial version. You must also show how the performance of your program changes as you:

- Increase the number of threads in the pool
- Increase the total amount of work your program has to do.

In particular, we want to explore how the number of threads in the pool affects the performance, relative to the # of CPU core you have. You also want to explore how the number of individual tasks affects the performance. For example, given a specific total number of iterations, is better to run a small number of long-running tasks, or a large number of smaller tasks? How would you be able to tell this from looking at the timings of your program?

To assess the run time/speedup, you need serial benchmarks and parallel benchmarks for different inputs. You can do something like this following:

- Run the serial program for computing π to a fixed number of iterations, e.g. 20,000,000,000
 - Repeat the execution multiple times for this number of iterations. This will give you a larger, more generalizable number of samples.
 - Do this for more/fewer iterations, so you can plot the serial run time as a function of number of iterations
- Run the program with a fixed number of threads and the total number of tasks that also add to 20,000,000,000, e.g. 5 tasks with 4,000,000,000 iterations each.
- Create multiple runs and vary the data. For example, do some runs when
 - # of worker threads is < the number of cores on your system
 - # of worker threads is = the number of cores on your system
 - # of worker threads is > the number of cores on your system
 - # of tasks is < # of # of worker threads
 - # of tasks is = # of # of worker threads
 - # of tasks is > # of # of worker threads

Also, remember that for any given set of arguments, the run time will always vary a bit. So make sure that for each set of inputs (i.e. each specific input file) you run at least 30 repetitions. You will need to record the timing from each repetition and then do some basic analysis - visualize the data in R and look at mean/median time (are they even similar?). Create a script for running the tests and generating files with test data.

Your report will need to include all the information you need to discuss your program performance - descriptions of the tests that you ran (and # of repetitions for each test), what aspect of your program's performance these tests were meant to assess, what the data summaries were (e.g. the histograms, plots of observed speedup vs theoretical speedup, etc.), and what conclusions you can draw from what you have observed.

4 Grading

Your code must compile and run on the Docker container provided to you. We will compile and grade your code using the same Docker container.

You need to provide all the necessary source code (C code, Makefile, and scripts), test cases, and input files. We will use our own input files as well. We will also use `valgrind` with and without DRD to check your code for run-time errors.

Grade breakdown:

- Working code that implements the parallelized system described above: **50 marks**
- Report with all the analysis: **50 marks**
 - Correctness: **10 marks**
 - Performance **40 marks**

Deductions will include, but will not be limited to:

- Compiler errors (code does not compile): **-50 marks (only the report is graded)**
- Compiler warnings: **-10 marks**
- DRD errors: **-10 marks**
- Memory leaks **-5 marks**
- Incorrect run-time behaviour, including
 - incorrect value of pi
 - incorrect/incomplete/missing thread pool
 - Deviations from the input/output specifications
 - etc.**up to -50 marks**
- Missing report components, including
 - Missing or incomplete analysis
 - Insufficient data to support the conclusions
 - Analysis that does not match the presented data
 - etc.**up to -50 marks**

Keep in mind that the report grading will take the team size into account. The report will take some time and effort to put together, and it will be easier for teams of two to present a more thorough and complete analysis of their program. As a result, reports submitted by teams of two will be graded more strictly than the reports submitted students working individually.